🚀 COURSE NAME: DevOps Bootcamp

Entering Learning Phase 3 - DevOps Core
🧱 Main Course Section: 8 - Build Automation & CI/CD with Jenkins

11 - Intro to Multibranch Pipeline [Branch name: feature/Intro-to-Multibranch-Pipeline]

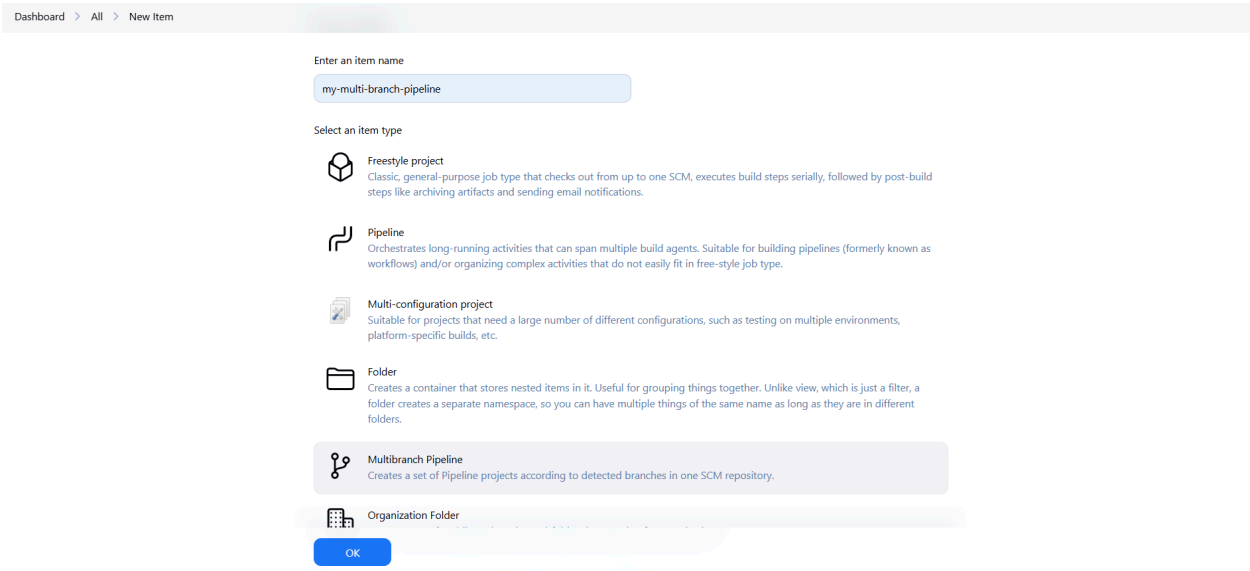
🗃️ Step 1: Creating a Jenkins Multibranch Pipeline with Git Integration

🔹 **Step Overview:**
In this step, we created a Jenkins Multibranch Pipeline project and integrated it with a Git repository. This provided insight into how Jenkins automates pipeline creation and execution for each branch in your repo, enabling efficient CI/CD management.

⚙️ Commands Executed:

📁 Accessing Jenkins Interface to Create a New Job:

bash
CopyEdit
```
# Navigating to Jenkins and selecting 'New Item'
http://<your-jenkins-server>:8080/view/all/newJob
```

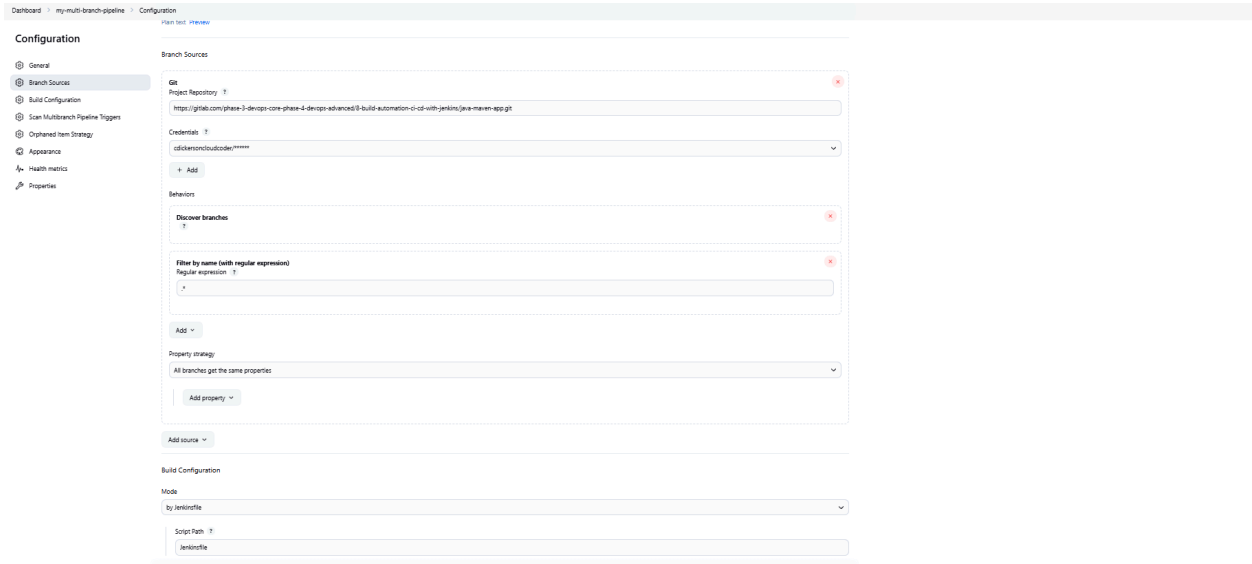


💡 This is where we initiate the creation of a new Jenkins job by choosing the *Multibranch Pipeline* option, which allows automatic branch detection and build configuration.

📂 Configuring Git as the Branch Source:

bash
CopyEdit

```
# Configuring Git as the source for branches
Branch Sources > Git > Project Repository
```



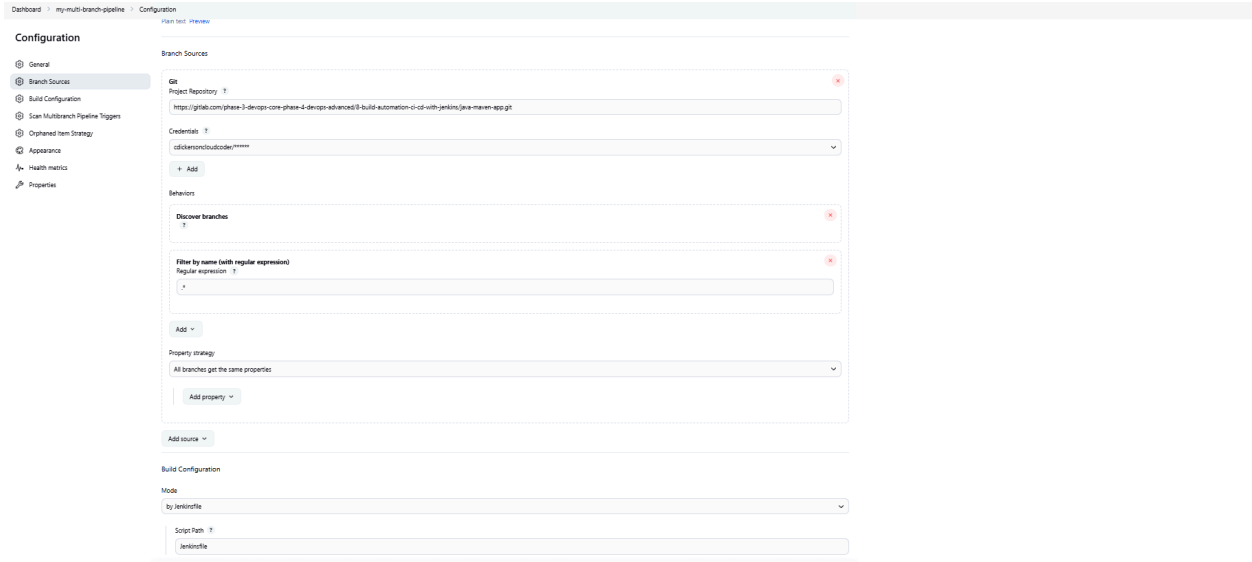
💡 This links Jenkins to your remote Git repository so it can scan for all branches and trigger jobs based on branch-specific Jenkinsfiles.

📂 Applying Branch Filters Using Regular Expressions:

bash
CopyEdit

```
# Using regex to include all branches
```

```
Filter by name (with regular expression): .*
```



💡 This ensures that Jenkins includes every branch in the repository when scanning, which is useful for complete multi-environment coverage in CI/CD pipelines.

📂 **Viewing the Pipeline Scan and Indexing Logs:**

bash
CopyEdit

```
# Viewing the indexing process
http://<your-jenkins-server>:8080/job/my-multi-branch-pipeline/indexing/console
```

Jenkins

Dashboard > my-multi-branch-pipeline > Scan Multibranch Pipeline

- Status
- Configure
- Scan Multibranch Pipeline Now
- Scan Multibranch Pipeline Log
  - View as plain text
- Multibranch Pipeline Events
- Delete Multibranch Pipeline
- Build History
- Project Relationship
- Check File Fingerprint
- Rename
- Config Files
- Pipeline Syntax
- Credentials

Build Queue (1)

part of my-multi-branch-pipeline » feature /payment #1

Build Executor Status 2/2

my-multi-branch-pipeline » starting-code #1 (Build)

my-multi-branch-pipeline » jenkins-jobs #1 (part)

my-multi-branch-pipeline » starting-code #1

my-multi-branch-pipeline » jenkins-jobs #1

Scan Multibranch Pipeline Log

```
Started by user Chad Dickerson
[Thu May 22 15:59:07 UTC 2025] Starting branch indexing...
 > git --version # timeout=10
 > git --version # 'git version 2.39.5'
using GIT_ASKPASS to set credentials
 > git ls-remote --symref -- https://gitlab.com/phase-3-devops-core-phase-4-devops-advanced/8-build-automation-ci-cd-with-jenkins/java-maven-app.git # timeout=10
 > git rev-parse --resolve-git-dir /var/jenkins_home/caches/git-05aa04b31e410bd3b8cae425957393bf/.git # timeout=10
Setting origin to https://gitlab.com/phase-3-devops-core-phase-4-devops-advanced/8-build-automation-ci-cd-with-jenkins/java-maven-app.git
 > git config remote.origin.url https://gitlab.com/phase-3-devops-core-phase-4-devops-advanced/8-build-automation-ci-cd-with-jenkins/java-maven-app.git # timeout=10
Fetching & pruning origin...
Listing remote references...
 > git config --get remote.origin.url # timeout=10
 > git --version # timeout=10
 > git --version # 'git version 2.39.5'
using GIT_ASKPASS to set credentials
 > git ls-remote -h -- https://gitlab.com/phase-3-devops-core-phase-4-devops-advanced/8-build-automation-ci-cd-with-jenkins/java-maven-app.git # timeout=10
Fetching upstream changes from origin
 > git config --get remote.origin.url # timeout=10
using GIT_ASKPASS to set credentials
 > git fetch --tags --force --progress --prune -- origin +refs/heads/*:refs/remotes/origin/* # timeout=10
Checking branches...
  Checking branch starting-code
      'Jenkinsfile' found
    Met criteria
Scheduled build for branch: starting-code
  Checking branch jenkins-jobs
      'Jenkinsfile' found
    Met criteria
Scheduled build for branch: jenkins-jobs
  Checking branch feature/payment
      'Jenkinsfile' found
    Met criteria
Scheduled build for branch: feature/payment
  Checking branch jenkins-shared-lib
      'Jenkinsfile' found
    Met criteria
Scheduled build for branch: jenkins-shared-lib
  Checking branch master
      'Jenkinsfile' found
    Met criteria
Scheduled build for branch: master
Processed 5 branches
[Thu May 22 15:59:10 UTC 2025] Finished branch indexing. Indexing took 3.1 sec
Finished: SUCCESS
```

Jenkins

Dashboard > my-multi-branch-pipeline >

- Status
- Configure
- Scan Multibranch Pipeline Now
- Scan Multibranch Pipeline Log
- Multibranch Pipeline Events
- Delete Multibranch Pipeline
- Build History
- Project Relationship
- Check File Fingerprint
- Rename
- Config Files
- Pipeline Syntax
- Credentials

Build Queue

No builds in the queue.

Build Executor Status 0/2

my-multi-branch-pipeline

Branches (5)

S	W	Name ↓	Last Success
✓	☼	feature/payment	50 sec #1
✓	☼	jenkins-jobs	50 sec #1
✗	☁	jenkins-shared-lib	N/A
✗	☁	master	N/A
✗	☁	starting-code	N/A

Icon: S M L

💡 This log shows how Jenkins scans your Git repo, identifies branches, and sets up pipelines automatically using Jenkinsfiles found in each branch.

🔍 **What We Learned:**

- Jenkins Multibranch Pipelines allow scalable and automated CI/CD for multiple branches.
- Git integration enables Jenkins to dynamically detect branches and build pipelines accordingly.
- Regular expressions can be used to filter branches effectively during pipeline scans.

✅ **Summary:**
You've just configured a Jenkins Multibranch Pipeline integrated with a Git repository. This adds to your understanding of **Jenkins CI/CD automation** by showing how to automatically scan, configure, and execute pipelines based on branch structures. 🚀

🚀 Step 2: Jenkinsfile Configuration for Multibranch Pipeline Execution

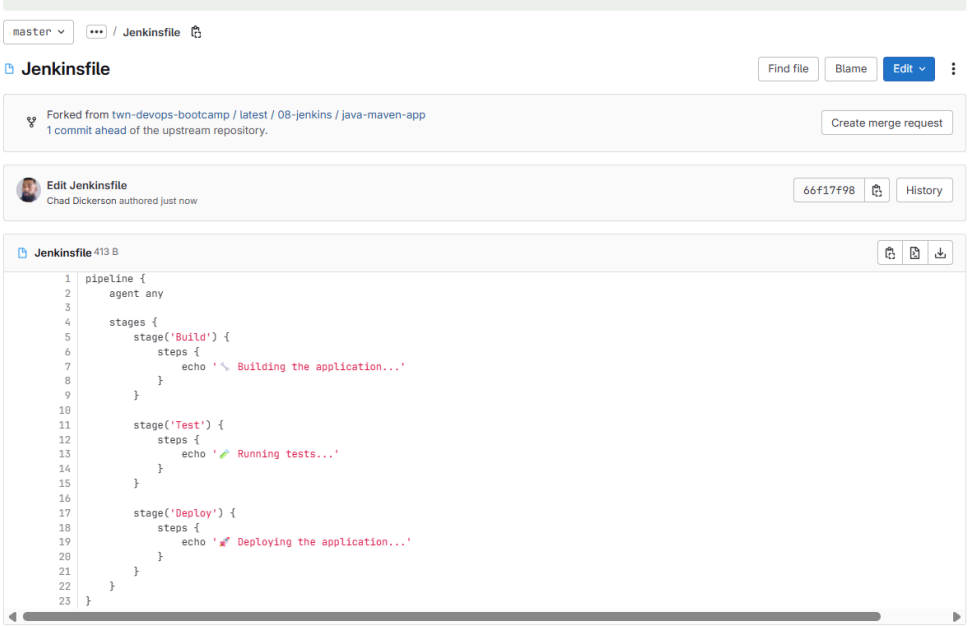
🔹 **Step Overview:**
In this step, we configured a `Jenkinsfile` in the GitLab repository to define stages for testing, building, and deploying a Java Maven application. This provided insight into how Jenkins executes conditional logic within pipelines based on the active branch—essential for robust CI/CD practices.

⚙️ **Commands Executed:**

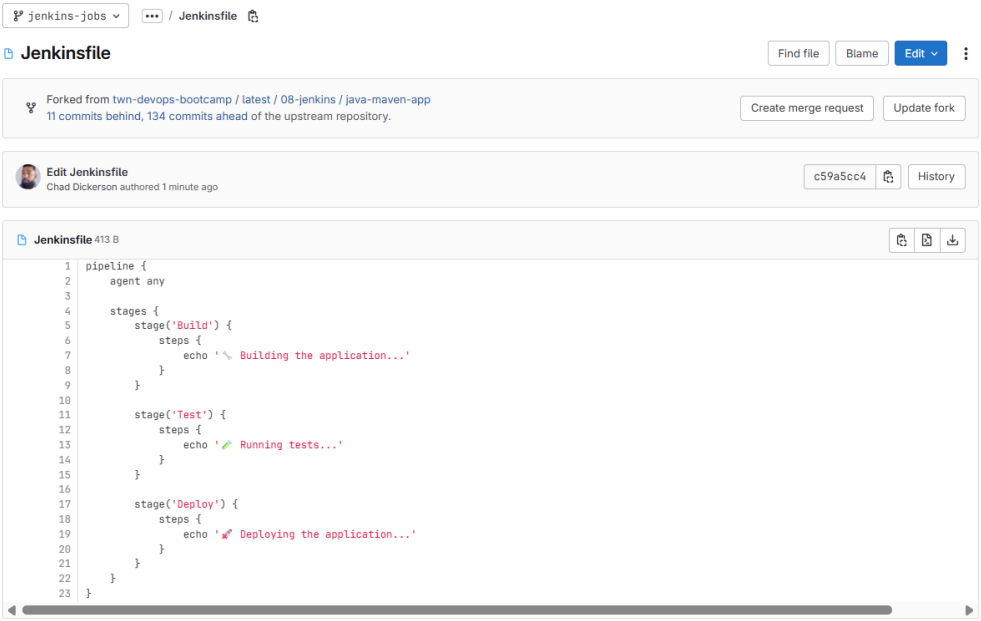
📁 **Accessing the Jenkinsfile in GitLab:**

bash
CopyEdit
```
# GitLab Repository Jenkinsfile (master & jenkins-jobs branches)
```

https://gitlab.com/.../java-maven-app/-/blob/master/Jenkinsfile



https://gitlab.com/.../java-maven-app/-/blob/jenkins-jobs/Jenkinsfile



💡 This file defines pipeline behavior. Branch-specific logic ensures that only `master` triggers build and deployment stages.

📂 **Jenkinsfile Configuration Highlights:**

groovy

CopyEdit

```
pipeline {
    agent any

    environment {
        BRANCH_NAME = "${env.BRANCH_NAME}"
    }

    stages {
        stage('Test') {
            steps {
                echo '🧪 Running tests...'
                echo "Executing pipeline for branch: ${BRANCH_NAME}"
            }
        }

        stage('Build') {
            when {
                expression {
                    return env.BRANCH_NAME == 'master'
                }
            }
            steps {
                echo '🔧 Building the application...'
                sh 'echo Build step running...'
            }
        }

        stage('Deploy') {
            when {
                expression {
                    return env.BRANCH_NAME == 'master'
                }
            }
            steps {
                echo '🚀 Deploying the application...'
                sh 'echo Deploy step running...'
            }
        }
```

```
    }
}
```

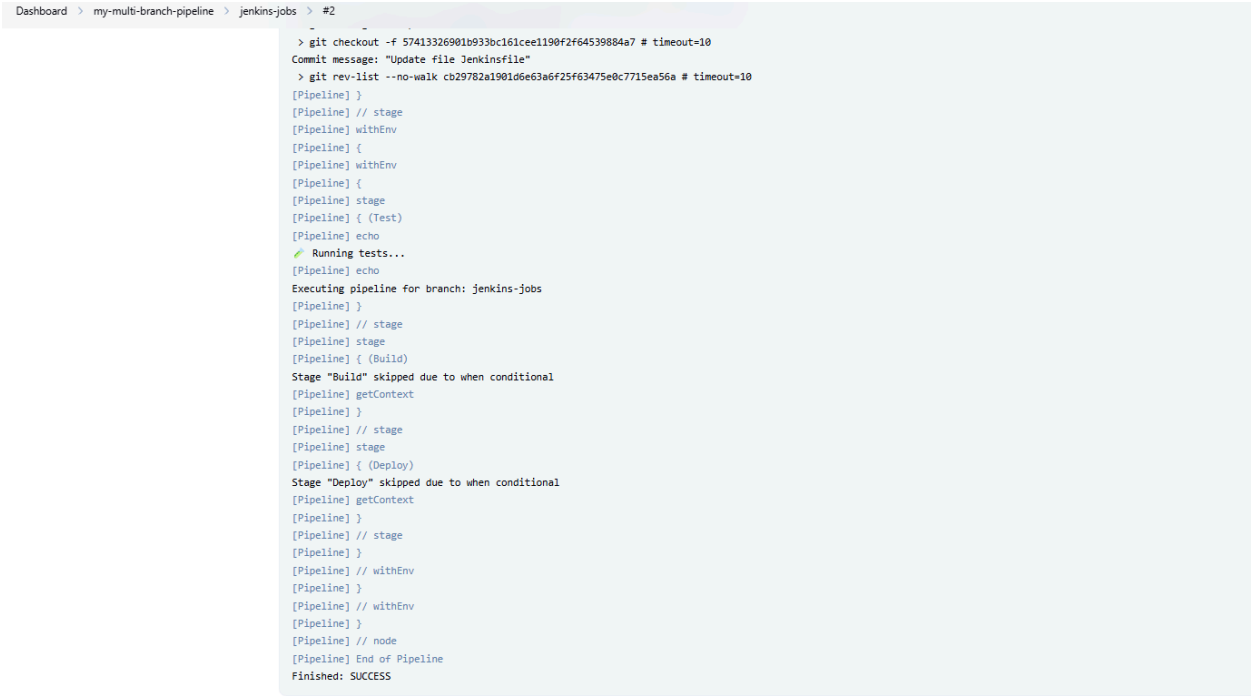
💡 This configuration uses conditional logic with `when` to ensure build and deploy steps only run on the `master` branch. All branches still run the 'Test' stage.

📂 Triggering the Jenkins Scan for Pipeline Updates:

bash
CopyEdit

```
# Scanning the Multibranch Pipeline in Jenkins
```

`http://<your-jenkins-server>:8080/job/my-multi-branch-pipeline/`



💡 This triggers Jenkins to re-index the repository, detecting any updates to branch Jenkinsfiles.

📂 **Viewing Console Output of Branch Builds:**

bash
CopyEdit
```
# Viewing pipeline execution logs
```

http://<your-jenkins-server>:8080/job/my-multi-branch-pipeline/job/jenkins-jobs/2/console

```
Dashboard > my-multi-branch-pipeline > jenkins-jobs > #2

[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git
using credential gitlab-credentials
 > git rev-parse --resolve-git-dir /var/jenkins_home/workspace/lti-branch-pipeline_jenkins-jobs/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://gitlab.com/phase-3-devops-core-phase-4-devops-advanced/8-build-automation-ci-cd-with-jenkins/java-maven-app.git # timeout=10
Fetching without tags
Fetching upstream changes from https://gitlab.com/phase-3-devops-core-phase-4-devops-advanced/8-build-automation-ci-cd-with-jenkins/java-maven-app.git
 > git --version # timeout=10
 > git --version # 'git version 2.39.5'
using GIT_ASKPASS to set credentials
 > git fetch --no-tags --force --progress -- https://gitlab.com/phase-3-devops-core-phase-4-devops-advanced/8-build-automation-ci-cd-with-jenkins/java-maven-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
Checking out Revision 5741332690b933bc161cee1190f2f6453988a47 (jenkins-jobs)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 5741332690b933bc161cee1190f2f6453988a47 # timeout=10
Commit message: "Update file Jenkinsfile"
 > git rev-list --no-walk cb29782a1901d6e63a6f25f63475e0c7715ea56a # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
🧪 Running tests...
[Pipeline] echo
Executing pipeline for branch: jenkins-jobs
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
Stage "Build" skipped due to when conditional
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
Stage "Deploy" skipped due to when conditional
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

http://<your-jenkins-server>:8080/job/my-multi-branch-pipeline/job/master/2/console

```
Dashboard > my-multi-branch-pipeline > master > #2

 > git --version # timeout=10
 > git --version # 'git version 2.39.5'
using GIT_ASKPASS to set credentials
 > git fetch --no-tags --force --progress -- https://gitlab.com/phase-3-devops-core-phase-4-devops-advanced/8-build-automation-ci-cd-with-jenkins/java-maven-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git config remote.origin.url https://gitlab.com/phase-3-devops-core-phase-4-devops-advanced/8-build-automation-ci-cd-with-jenkins/java-maven-app.git # timeout=10
 > git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
Checking out Revision 54078f67b789f69ca14e90b78f23c9d7015f9193 (master)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 54078f67b789f69ca14e90b78f23c9d7015f9193 # timeout=10
Commit message: "Edit Jenkinsfile"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
🧪 Running tests...
[Pipeline] echo
Executing pipeline for branch: master
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
🔨 Building the application...
[Pipeline] sh
+ echo Build step running...
Build step running...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
🚀 Deploying the application...
[Pipeline] sh
+ echo Deploy step running...
Deploy step running...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

💡 These logs show which stages were executed per branch, verifying conditional stage logic (e.g., only `Test` runs on feature branches, while `Build` & `Deploy` run on `master`).

🔍 **What We Learned:**

- `Jenkinsfile` controls the pipeline stages and their execution flow.
- Conditional logic using `when` enables dynamic behavior based on branch names.
- Multibranch Pipelines allow different workflows for different branches—supporting flexible CI/CD strategies.

✅ **Summary:**
You've just created and configured a branch-aware `Jenkinsfile` to define a flexible CI/CD pipeline. This adds to your understanding of **Jenkins automation** by showing how to **conditionally execute pipeline stages** based on branch context. 🚀